DEPARTMENT OF COMPUTER ENGINEERING • B.E. SEMESTER VII • DEVOPS LAB • VIDYALANKAR INSTITUTE OF TECHNOLOGY

DevOps-Enabled Venue Resource Booking System (VRBS)

An automated path from a Git commit to a running, tested, and recoverable enterprise deployment.

Rishi Phadale
Roll No. 23102C0070 • Division C
Guide: Dr. Snehal Andhare

1 THE PROBLEM

Manual campus venue scheduling relies on spreadsheets and paper forms, causing double-bookings, booking collisions, and lack of real-time slot visibility across facilities.

- ✕ No real-time visibility into slot availability across auditoriums, seminar halls, and computer labs.
- ✕ Paper tokens and manual approvals cause scheduling conflicts and duplicate venue allocations.
- ✕ Risky manual deployments on production servers with prolonged downtime and no automated rollback.

2 THE SOLUTION: VRBS WEB APP (MVP)

`FR-01` User & Admin Auth (RBAC)	`FR-02` Multi-Venue Directory & Cap	`FR-03` Flexible Slots (08:00–20:00)
`FR-04` Conflict Prevention Engine	`FR-05` Enforced State Machine	`FR-06` Self-Service Booking Hub
`FR-07` Public Schedule Matrix	`FR-08` Admin Approval & Audit	`FR-09` Health Gate /actuator/health

Goal: Keep the application focused and make the DevOps lifecycle around it the core deliverable: Git branching, Jenkins CI/CD automation, a Selenium quality gate, Docker container packaging, and Ansible IaC provisioning with one-command rollback. Includes dynamic hourly slot grids and instant double-booking prevention.

3 END-TO-END CI/CD PIPELINE: COMMIT → PRODUCTION

`Jenkins Build #12 • All Stages Green • 2 min 08 s End-to-End`

CODE	STAGE 1	STAGE 2	QUALITY GATE	STAGE 3	STAGE 4	STAGE 5	STAGE 6	STAGE 7
Commit & PR	Checkout	Compile & Test	Selenium E2E	Package WAR	Build Image	Publish	Deploy Fresh	Health Gate
`GIT • GITHUB`	`JENKINS SCM`	`MAVEN • JUNIT 5`	`HEADLESS CHROME`	`MAVEN PACKAGE`	`DOCKER BUILD`	`DOCKER HUB`	`DOCKER RUN`	`SPRING ACTUATOR`
feature/* → develop → main branch merge	Webhook pulls clean branch & Jenkinsfile	115 tests executed on in-memory H2 DB	10 browser journeys verified. Fail = STOP	Production venue-booking.war artifact	venue-booking:${BUILD} + :latest tag	Push versioned image to registry	Target VM node: 8081→8081 port binding	curl /actuator/health == UP verification

ON FAILURE: Build turns red immediately; packaging and container deployment are completely halted. JUnit reports and Selenium failure screenshots are automatically archived. Broken builds never reach production users.

ANSIBLE IAC: `playbook.yml` configures packages, users, firewall, and deploys the container. Re-running yields `changed=0` (idempotent). `rollback.yml` restores the previous stable release container in 1 automated step.

4 APPLICATION ARCHITECTURE

UI Layer:	Thymeleaf, Bootstrap 5, JS Slot Grid
Controllers:	REST API (12 endpoints) + Spring MVC
Service:	BookingService, ConflictEngine, UserService
Repository:	Spring Data JPA: Venue, Booking, User
Database:	H2 in-memory (CI test) • MySQL 8 (Production)

BOOKING STATUS STATE MACHINE (FR-05)
`PENDING → APPROVED → IN_PROGRESS → COMPLETED`
Also `REJECTED` / `CANCELLED`. Terminal states are immutable (IllegalStateException).

5 AUTOMATED TESTING

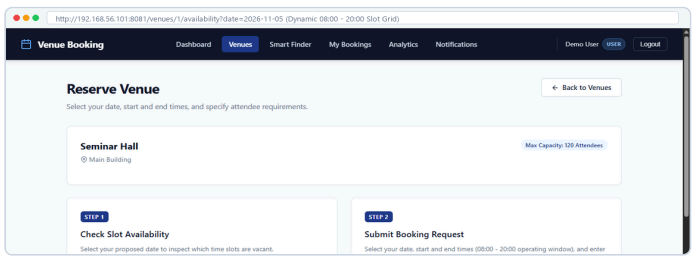
18	10	1
Service / Logic	Selenium E2E	Spring Smoke
JUnit 5	Headless Chrome	Context Load

- ✕ Zero-conflict guarantee: overlapping slot requests rejected.
- ✕ Complete state machine and terminal-state immutability tests.
- ✕ Full UI journeys: Login → Venue picker → Time slot → Booking.
- ✕ Headless browser failure screenshots archived by Jenkins.

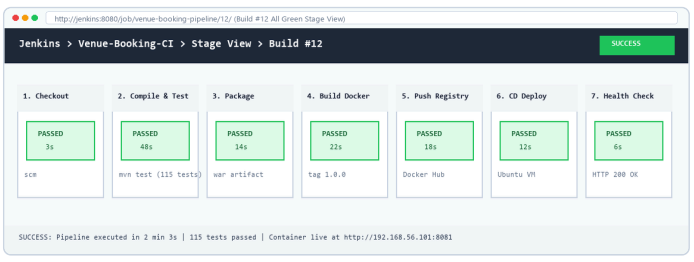
6 INFRASTRUCTURE AS CODE

- ✕ Packages: OpenJDK 17, Docker CE, curl, net-tools, ufw.
- ✕ Unprivileged system user & secure directory permissions.
- ✕ Jinja2 templates: application environment & Docker config.
- ✕ Automated rollback restores last stable release container.

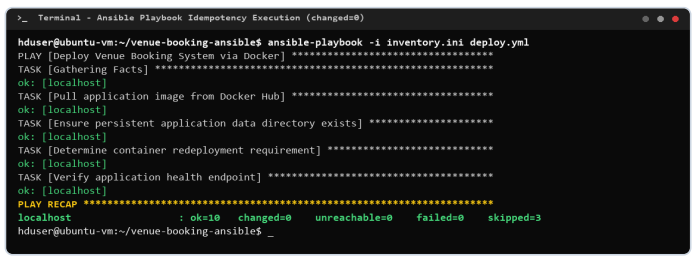
```
1st run:  ok=15  changed=8  failed=0
re-run:   ok=14  changed=0  failed=0  ← idempotent
rollback.yml → stop • restore backup • restart • verify
```



Live App on localhost:8085: Venue catalogue, flexible slot matrix (08:00–20:00), zero-collision conflict prevention.



Jenkins Stage View, Build #12: Every stage from Checkout to Actuator Health Check passes with zero regressions.



Ansible Re-run (Idempotency): `changed=0 failed=0` proves the node is already in the desired state.

15	8	29	2m 08s	0	1-Step
Curriculum Tasks Completed	Automated Jenkins Stages	Automated Tests (Unit + E2E)	Commit to Deployed Container	Changes on Re-run (Idempotent)	Automated Rollback to Stable

7 15-TASK DEVOPS CURRICULUM JOURNEY

PLAN	CODE & COLLAB	INTEGRATE	TEST	DELIVER	OPERATE & RELEASE
1 Problem & Scope	4 Repo & Branch Policy	7 Jenkins CI Job	9 Selenium Suite	11 Docker Lifecycle	13 Ansible Playbook
2 Agile Planning	5 Feature Branch & PR	8 Jenkinsfile + Tomcat	10 Testing in Jenkins	12 Jenkins → Docker	14 Idempotency & Rollback
3 Arch & Setup	6 Merge Conflict & v1.0.0				15 Final Release & Viva

8 TECHNOLOGY STACK

`BACKEND` Java 17 • Spring Boot 3	`BUILD` Maven 3.9 → WAR
`UI` Thymeleaf • Bootstrap 5	`DATABASE` H2 (CI) • MySQL 8 (Prod)
`TESTING` JUnit 5 • Selenium 4	`CI/CD` Jenkins Declarative
`CONTAINER` Docker CE • Docker Hub	`IAC` Ansible • Jinja2

9 KEY DEVOPS CONCEPTS SHOWN

Continuous Integration: Every push is compiled and verified through 115 tests.

Continuous Delivery: Verified artifacts automatically packaged as Docker images.

Pipeline as Code: Versioned `Jenkinsfile` controls entire delivery lifecycle.

Quality Gate: Broken tests stop deployment; defect-free releases guaranteed.

IaC & Idempotency: Repeatable Ansible runs converge safely with `changed=0`.

10 LEARNINGS & FUTURE SCOPE

Fast Feedback: Regressions are caught in 28 seconds before reaching users.

Automated Recovery: Rollback script restores previous stable release in 1 command.

Production Hardening: Secure systemd isolation and health check probes.

Future Work: Kubernetes orchestration (K8s/Helm), Prometheus & Grafana monitoring, and OAuth2/SSO campus identity integration.